

A Comprehensive Formal Security Analysis of OPC UA with ProVerif

Steve Kremer¹

Usenix Security'25

joint work with Vincent Diemunsch^{1,2} and Lucca Hirschi¹

¹Centre Inria de l'Université de Lorraine & LORIA ²ANSSI

25 years of ProVerif



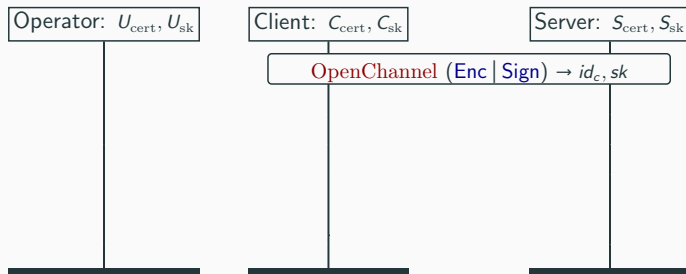
Communication protocol for Industrial Control Systems (ICS)

- Secure command and control of industrial automata
- Deployed in critical infrastructures (power grids, oil and gas facilities, water treatment, etc.)
- Standardized by the OPC Foundation, v. 1.04 standardized as IEC 62541
- High interest by security agencies (ANSSI, BSI, CISA)



security aspects span 5 documents (out of 20+),
totaling 528 pages

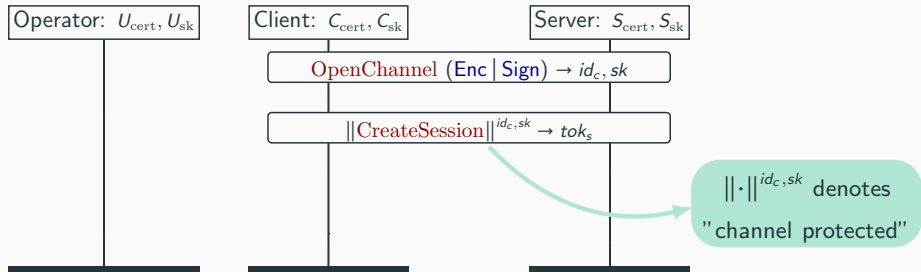
A high-level overview of OPC UA



Open a secure channel:

- Security Policy: either PKE(RSA)- or DH(ECC)-based (v.1.05) protocol
- Mode:
 - either integrity protection only (**Sign**), or
 - also confidentiality (**Enc**), or
 - None

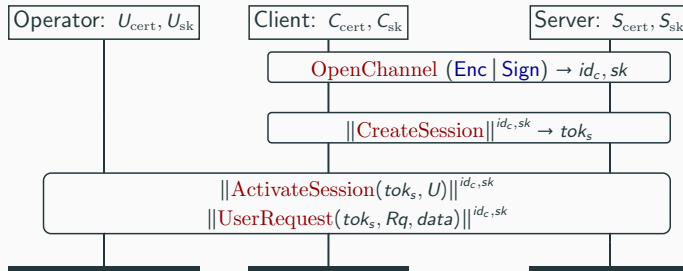
A high-level overview of OPC UA



Create Session:

- Session Security Level : SSec (server + client certificates) or SNoAA (No Application Authentication)

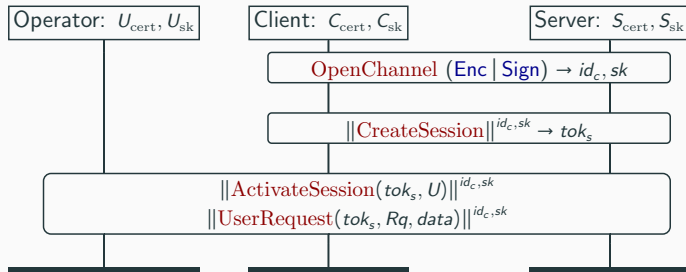
A high-level overview of OPC UA



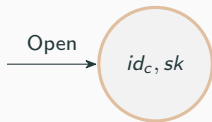
User authentication:

- Certificate-based, username/password, or anonymous

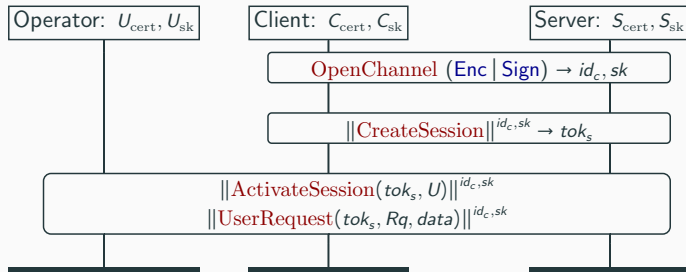
A high-level overview of OPC UA



State machine:



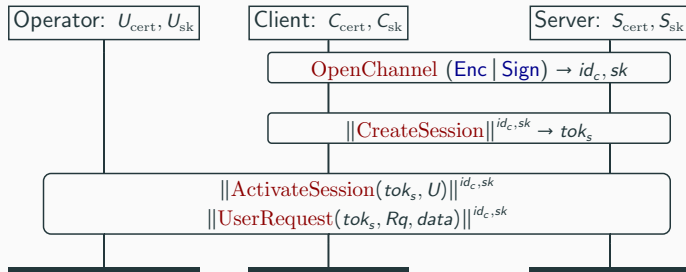
A high-level overview of OPC UA



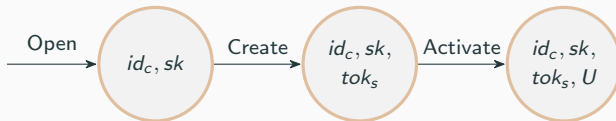
State machine:



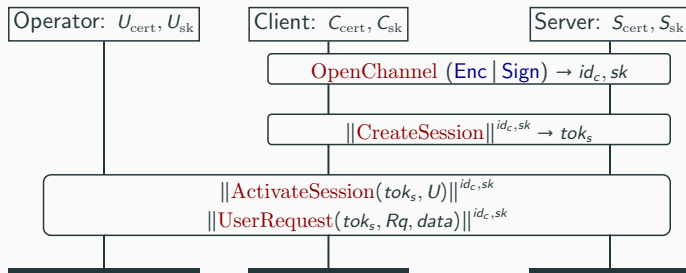
A high-level overview of OPC UA



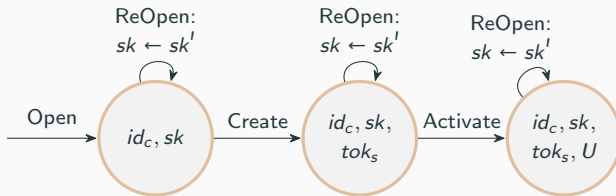
State machine:



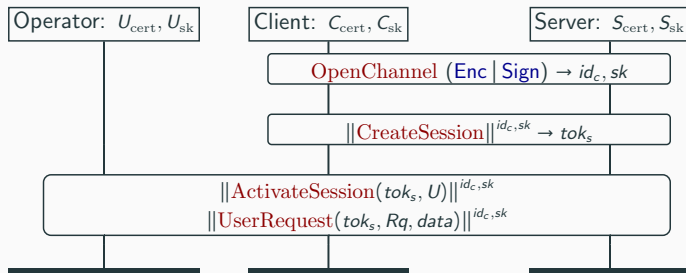
A high-level overview of OPC UA



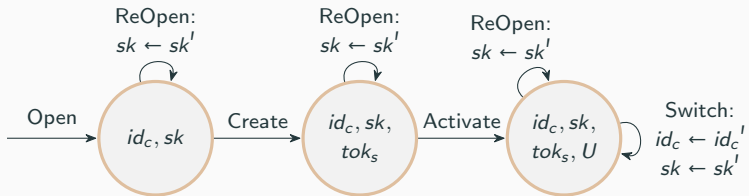
State machine:



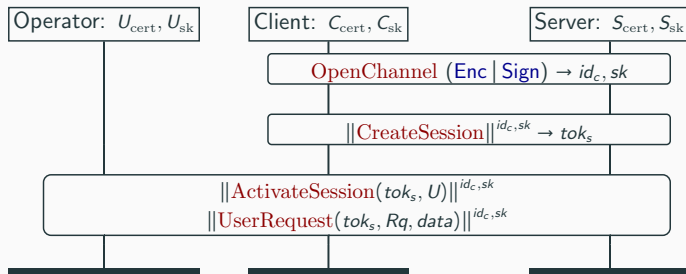
A high-level overview of OPC UA



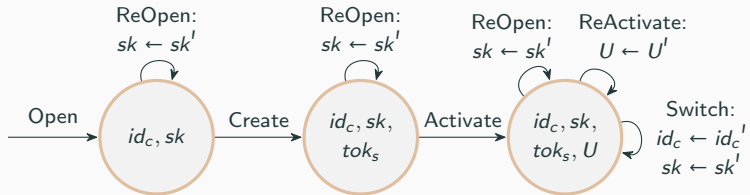
State machine:



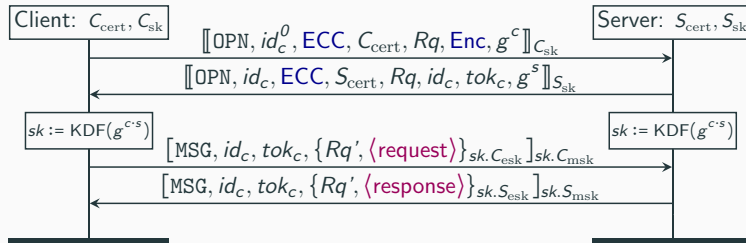
A high-level overview of OPC UA



State machine:

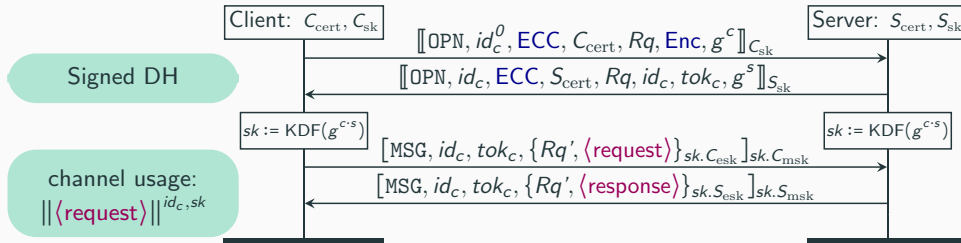


A closer look at the OpenChannel protocol : ECC + Enc



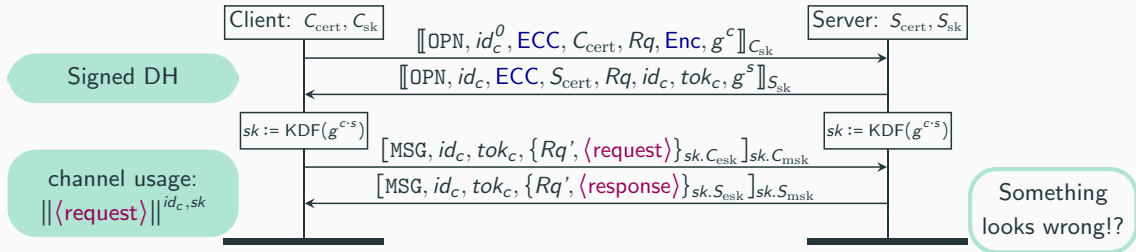
Notations: signature: $\llbracket \cdot \rrbracket$. mac: $[\cdot]$. sym. encryption: $\{ \cdot \}$.

A closer look at the OpenChannel protocol : ECC + Enc



Notations: signature: $\llbracket \cdot \rrbracket$. mac: $[\cdot]$. sym. encryption: $\{\cdot\}$.

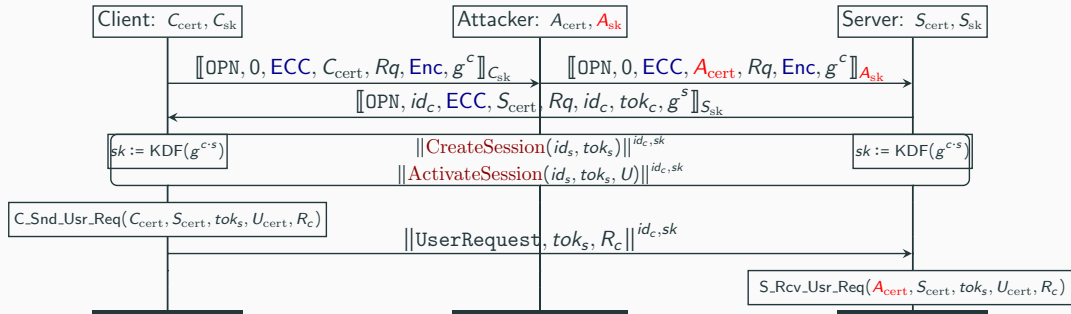
A closer look at the OpenChannel protocol : ECC + Enc



Notations: signature: $\llbracket \cdot \rrbracket$. mac: $[\cdot]$. sym. encryption: $\{\cdot\}$.

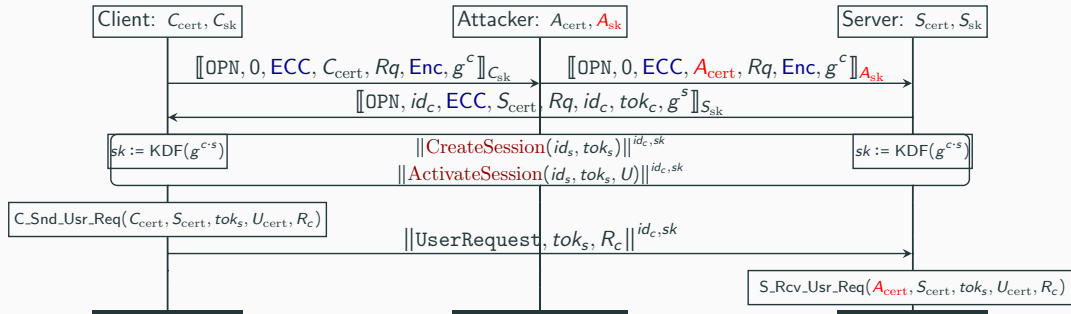


Client impersonation attack: Gavin strikes back!



Attack: S believes $UserRequest$ stems from U on client A while it originates from U on client C .

Client impersonation attack: Gavin strikes back!



Attack: S believes $UserRequest$ stems from U on client A while it originates from U on client C .

Take away: We were able to find a 30 year old attack with a 25 year old tool :-)

A comprehensive model of OPC UA

Protocol model:

- all subprotocols (OpenChannel, CreateSession, ActivateSession, UserRequest)
- all security policies, channel modes, session security levels, user credentials

A comprehensive model of OPC UA

Protocol model:

- all subprotocols (OpenChannel, CreateSession, ActivateSession, UserRequest)
- all security policies, channel modes, session security levels, user credentials

Threat model:

- standard Dolev-Yao attacker
- compromise of either long-term keys, or channel keys

A comprehensive model of OPC UA

Protocol model:

- all subprotocols (OpenChannel, CreateSession, ActivateSession, UserRequest)
- all security policies, channel modes, session security levels, user credentials

Threat model:

- standard Dolev-Yao attacker
- compromise of either long-term keys, or channel keys

Size of the model:

- unfolded process generated: 8.6k LoC of applied π -calculus
- translated into 2.3k initial Horn clauses
- e.g., more LoC and Horn clauses than TLS 1.3 in [Bhargavan et al., CCS 2022]

A comprehensive model of OPC UA

Protocol model:

- all subprotocols (OpenChannel, CreateSession, ActivateSession, UserRequest)
- all security policies, channel modes, session security levels, user credentials

Threat model:

- standard Dolev-Yao attacker
- compromise of either long-term keys, or channel keys

Size of the model:

- unfolded process generated: 8.6k LoC of applied π -calculus
- translated into 2.3k initial Horn clauses
- e.g., more LoC and Horn clauses than TLS 1.3 in [Bhargavan et al., CCS 2022]

Main security properties: (Perfect Forward) Secrecy and Authentication of requests

Need for modularity and tooling

Generate protocol models for:

- different protocol configurations:
cryptography, channel security modes, supported user credentials, session security level
- threat models:
compromise of long-term keys, or channel keys
- simplified versions:
disable channel reopening, or channel switching

⇒ combinations result in **7056 configurations**

Need for modularity and tooling

Generate protocol models for:

- different protocol configurations:
cryptography, channel security modes, supported user credentials, session security level
- threat models:
compromise of long-term keys, or channel keys
- simplified versions:
disable channel reopening, or channel switching

⇒ combinations result in **7056 configurations**

```
prove -c "ECC,Sign|Enc,reopen,  
SSec,pwd|cert,switch,lt_leaks"
```

Need for modularity and tooling

Generate protocol models for:

- different protocol configurations:
cryptography, channel security modes, supported user credentials, session security level
- threat models:
compromise of long-term keys, or channel keys
- simplified versions:
disable channel reopening, or channel switching

```
prove -c "ECC,Sign|Enc,reopen,  
SSec,pwd|cert,switch,lt_leaks"
```

⇒ combinations result in **7056 configurations**

Configurations organized in a lattice

⇒ efficient exploration, identification of minimal attack configurations / maximal secure configurations

Spoiler: ~~we~~ ProVerif found attacks!

Name	Violated properties	Threat	Config.	Impact	Root Causes	Remediations: Fix, Mitigation and Enhancement
Race Condition for User Contexts	$\text{Agr}_{\text{CS}}[U]$: S receives a request from C , U_2 but U_1 sent it	$\emptyset$	$\emptyset$	Abuse of user rights, wrong logging (user)	• RC1: missing binding btw user and activated session (tok_s)	• Fix1: change tok_s at each user activation • Mit1: limit activations to users with same rights
Client Impersonation in ECC	$\text{Agr}_{\text{CS}}[C]$: S receives a request from C^a , U but U sent it on C	$\text{Att}(C_{sk}^a)$	ECC + SNoAA	Server confusion (C), wrong logging	• RC2: lack of receiver identity in OPN requests and responses	• Fix2: enforce receiver identity in ECC • Enh2: enforce stronger checks in SNoAA
KCI: User Impersonation	$\text{Agr}_S^{-0}[U]$: S receives a request from C^a , U but user U on C did not send it	$\text{Att}(C_{sk}^a)$ $\text{Att}(S_{sk})$	Cert	User impersonation	• RC3: lack of binding of user's authentication (user's signature) to full context (client identity)	• Fix3: add client's identity in user's signature • Enh3: plan to release some enhanced security versions of the user tokens
Downgrade of Password Secrecy	Conf_{Pwd} : Compromise of user password	$\text{Att}(sk)$	Enc + Pwd	Stealing of user password, full user impersonation	• RC4: relaxed security configuration to avoid what may appear redundant password encryption	• Fix4: use a challenge response mechanism for password authentication or a PAKE protocol • Enh4: dedicated password encryption in Enc
Risk of Signature Oracle	Conf_C , Conf_{Pwd} , Agr_C : Server impersonation towards (C , U)	$\emptyset$	ECC + SNoAA	Server impersonation (password stealings)	• RC5: lack of context in signatures	• Fix5: add context and tags to the signature • Enh5: check certificates and nonce lengths
Session Hijack • Reopen (• Switch)	$\text{Agr}_S^{-1}[U]$: S receives a request from C^a , U but user U on C did not send it	$\text{Att}(C_{sk})$	Sign + Reopen	Replay (impersonation)	• RC6: secure channel takeover at key renewal (Reopen) • RC7: no session ownership proof	• Fix6: linking channel secret keys through renewal • Fix7: keep tok_s secret even is Sign mode, and use it as a MAC computed on each request
KCI: Session and User Confusion	$\text{Agr}_S^{-1}[U]$: S receives a request from C , U but it was sent by a dummy user U^d on C	$\text{Att}(S_{sk})$	Reopen	User impersonation	• RC6: secure channel takeover at key renewal (Reopen) • RC7: no session ownership proof	• Fix6: linking channel secret keys through renewal • Fix7: keep tok_s secret even is Sign mode, and use it as a MAC computed on each request

Toil, Tears and Sweat (1)

- **Non-termination** on most non-trivial configurations



```
get C_sessions(...,chan(=C_cert, =S_cert, =mode, =ch_s, 0_keys: chkeys),...) in
...
(* key update: 0_keys -> sym_keys *)
insert C_sessions(...,chan(C_cert, S_cert, mode, ch_s, sym_keys),...)
```

Toil, Tears and Sweat (1)

- **Non-termination** on most non-trivial configurations



```
get C_sessions(...,chan(=C_cert, =S_cert, =mode, =ch_s, 0_keys: chkeys),...) in
...
(* key update: 0_keys -> sym_keys *)
insert C_sessions(...,chan(C_cert, S_cert, mode, ch_s, sym_keys),...)
```

- Add aggressive **noselect statements**.

```
noselect x1,x2:pkey, ..., x11:skey; table(C_sessions( *x1,[...],*x11))/9000
```

⇒ termination, but neither proof nor attack (Cannot be proved)

Toil, Tears and Sweat (1)

- **Non-termination** on most non-trivial configurations

```
get C_sessions(...,chan(=C_cert, =S_cert, =mode, =ch_s, 0_keys: chkeys),...) in
...
(* key update: 0_keys -> sym_keys *)
insert C_sessions(...,chan(C_cert, S_cert, mode, ch_s, sym_keys),...)
```

- Add aggressive **noselect statements**.

```
noselect x1,x2:pkey, ..., x11:skey; table(C_sessions( *x1,[...],*x11))/9000
```

⇒ termination, but neither proof nor attack (Cannot be proved)

- Iteratively add **lemmas** (>100)

```
let SK1 = keys(chtk1, C_enc_k1, C_sign_k, S_enc_k1, S_sign_k1) in
let SK2 = keys(chtk2, C_enc_k2, C_sign_k, S_enc_k2, S_sign_k2) in [...]
  event(insert_C_sessions(C_pk1, S_pk1, id1, SAtoken1, check1, SC1, [...]))
  && event(insert_C_sessions(C_pk2, S_pk2, id2, SAtoken2, check2, SC2, [...]))
==> C_enc_k1=C_enc_k2 && S_enc_k1=S_enc_k2 && S_sign_k1=S_sign_k2 && [...]].
```

2 entries with
same mac keys
⇒
same chan-
nel keys

Toil, Tears and Sweat (2)

Additional **ProVerif** wizardry:

- `new C_nonce[C_pk,ch]:nonce`
- `[precise],[ignoreAFewTimes],[induction]`
- `set removeEventsForLemma = false.`
- `set noselectIgnoreConclusion = false.`
(Do you know this one??)

Toil, Tears and Sweat (2)

Additional **ProVerif** wizardry:

- `new C_nonce[C_pk,ch]:nonce`
- `[precise],[ignoreAFewTimes],[induction]`
- `set removeEventsForLemma = false.`
- `set noselectIgnoreConclusion = false.`
(Do you know this one??)

commit breaks
termination



Here is a
new flag!

10 minutes later

Toil, Tears and Sweat (2)

Additional **ProVerif** wizardry:

- `new C_nonce[C_pk,ch]:nonce`
- `[precise],[ignoreAFewTimes],[induction]`
- `set removeEventsForLemma = false.`
- `set noselectIgnoreConclusion = false.`
(Do you know this one??)

commit breaks
termination

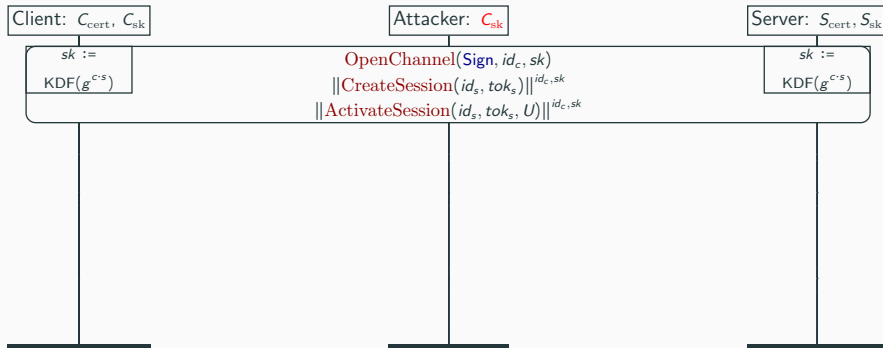


Here is a
new flag!

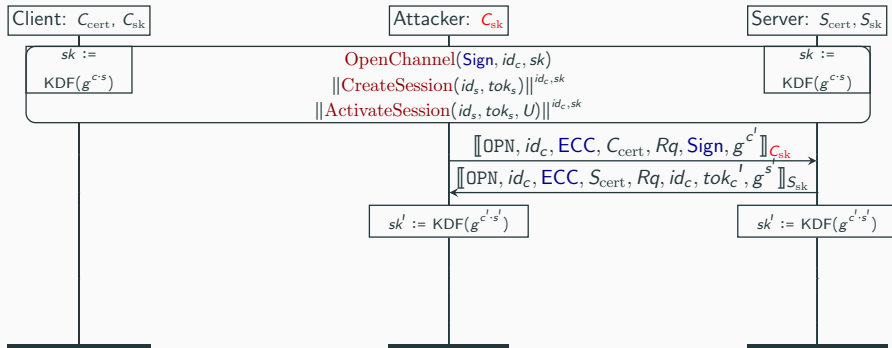
10 minutes later

Many options specific to a property / lemma
⇒ requires an important software engineering effort

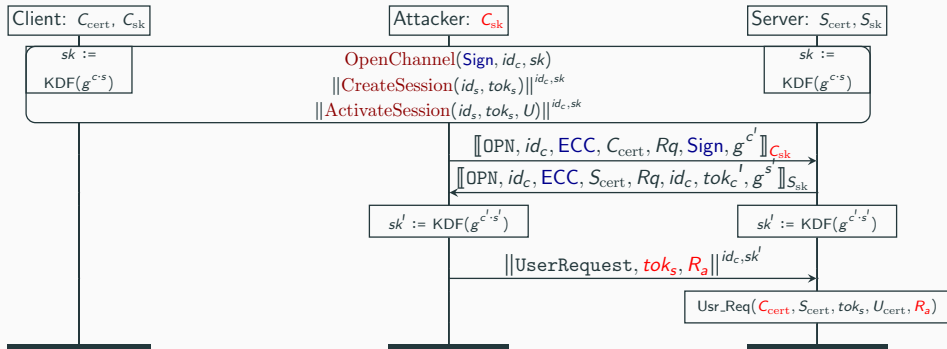
Session hijack by reopening a channel in Sign mode



Session hijack by reopening a channel in Sign mode



Session hijack by reopening a channel in Sign mode



Authentication violation: S receives R_a from (C_{cert}, U) but U did not send it.

- channel takeover: *lack of binding between channel keys before and after renewal*
- session hijack: *no ownership proof, tok_s is not secret*

5 attacks + 3 protocol weaknesses

- multi-layer nature of the protocol: channels, created and activated sessions, requests
- agility of the protocol due to OT constraints: switch/reopen channels, re-activate sessions, etc.
- complexity of the protocol: combination of multiple options and configurations

Main root cause: missing bindings between critical components and successive keys

- All findings responsibly disclosed to the OPC foundation between October 2023 and November 2024
- Attacks have been acknowledged but lengthy discussions about threat models
- Fixes proposed and discussed; some are still to be integrated
- May 2025: publication at Usenix and meeting with OPC foundation, ANSSI, BSI, CISA and FBI.
- Discussions about filing CVEs are still ongoing . . .

Conclusion

- Among the largest protocol models formally analyzed
 - comprehensive model
 - difficulty of proofs
- 5 attacks and 3 weaknesses found
- Tooling and proof methodology exploiting many of the latest ProVerif features
→ reusable for future analyses of complex protocols
- Starting point for proving security of fixes and future versions of OPC UA

Conclusion

- Among the largest protocol models formally analyzed
 - comprehensive model
 - difficulty of proofs
- 5 attacks and 3 weaknesses found
- Tooling and proof methodology exploiting many of the latest ProVerif features
→ reusable for future analyses of complex protocols
- Starting point for proving security of fixes and future versions of OPC UA

Happy Birthday ProVerif!